

AWS Certified Generative AI Developer — Professional

The Use Case

- Global law firm, 25 countries — case law, legal opinions, contracts, regulatory filings
- Requirements:** accurate context-aware answers, multi-jurisdictional research, strict client confidentiality, **citations to source documents on every response**, sub-second responses
- Outcomes:** research time down 70%, 42% more relevant precedents discovered, 92% recall, 95% of attorneys report higher confidence

Task 1.4 vs Task 1.5

Task	Focus
1.4	Where the vector store lives and how it's maintained — infrastructure, sharding, caching, sync
1.5	How good the retrieval is — chunking, embeddings, query handling, reranking. Recall and relevance.

1. Document Segmentation (Chunking)

Why legal chunking is uniquely hard: "The Party may terminate this Agreement, *provided that Section 5.1 conditions are met.*" Cut in the wrong place and the chunk says "Party may terminate" — the conditions vanish and the meaning inverts. Bad chunking produces bad legal advice.

Four approaches

Strategy	How it works
Fixed-size	Cut every N words. Simple and fast, but blind to structure — clauses break mid-sentence. Fine for plain narrative text.
Structure-aware	Cut on headings and paragraph boundaries , not word counts. Convert unstructured text to a structured format (e.g. HTML) first so <h1> , <h2> , <p> reveal logical divisions, then insert divider strings on those tags. Each chunk is a complete logical unit.
Hierarchical	Parent-child. Search the small child chunks, send the parent to the model. Precision from search, context from the parent. Built into Bedrock Knowledge Bases.
Semantic	Cut where the topic shifts, detected by similarity between consecutive sentences. Expensive (embeddings computed during chunking) but yields natural boundaries.

```
PARENT: Article 5 - Termination (2000 words)
+-- CHILD: 5.1 Conditions      (300 words)
+-- CHILD: 5.2 Notice period  (250 words)
+-- CHILD: 5.3 Effects        (400 words)
```

Bedrock Knowledge Bases chunking options

Option	Notes
Default	~300 tokens
Fixed-size	You set size and overlap
Hierarchical	Parent-child
Semantic	Meaning-based boundaries
No chunking	One file = one chunk, for small documents
Lambda custom	Triggered when a document lands in the S3 data source; your own logic

Transcript's approach — one strategy per document type

Case law	-> Bedrock chunking, hierarchy preserved
Internal opinions	-> Lambda structure-aware (headings)
Contracts / regs	-> Hierarchical (parent-child)

Exam-relevant: chunking strategy is chosen per document type, not once for the whole corpus.

Metadata and the practical reality

- Metadata (jurisdiction, practice area, precedential value) is attached **during** chunking — as in Task 1.4, adding it later means full re-

ingestion

- Metadata tags can be assigned from **important keywords** to mark logical section boundaries
- **Optimal chunking is found by trial and error.** Small chunks: better precision, broken context. Large chunks: full context, more noise. ~10% overlap protects meaning at the boundaries.

2. Embedding Solutions

Dimensionality

- **Amazon Titan Text Embeddings: 1,536 dimensions** — worth memorising
- Titan V2 allows choosing 1024, 512 or 256
- **More dimensions:** finer nuance, better similarity — but more storage, more RAM, slower search
- **Fewer dimensions:** cheaper and faster, some nuance lost

Dense vs sparse

Type	Characteristics
Dense	Every dimension carries a value. Higher dimensions give greater similarity fidelity. Captures meaning .
Sparse	Mostly zeros, lower effective dimensionality, more efficient. Behaves more like keyword matching.

Legal work needs both — which is what hybrid search provides.

Binary embeddings — cost optimisation

float32:	4 bytes per dimension	
binary:	1 bit per dimension	-> ~32x less storage

- Titan offers standard and **binary** embeddings to trade cost against accuracy
- OpenSearch Serverless supports **binary vectors and FP16** for optimised vector search
- Same principle as quantization in Task 1.4 — here the embedding model itself emits the compact form

Domain fit — more important than dimensionality

A general embedding model does not understand legal language. "Consideration" means something entirely different in contract law than in ordinary English; "party" is a legal entity, not a celebration.
--

General legal content	-> Titan Text Embeddings
Specialised practice areas	-> custom fine-tuned embedding models

Transcript result: **92% recall** on relevant precedents, well ahead of keyword search.

Multimodal and batch generation

- **Titan Multimodal Embeddings G1** places text and images in the **same vector space** — search images with text. Useful for scanned documents and diagrams.
- **Lambda in parallel** (transcript's approach) or **AWS Batch** for very large corpora
- Bedrock Knowledge Bases handles embedding generation automatically once you select the model

The same embedding model must be used for queries and documents. Different models mean different vector spaces and nothing will match . This is also why changing the embedding model forces a full corpus rebuild.

3. Vector Search Deployment

Option	Strength and when to choose
Bedrock Knowledge Bases	Fully managed — ingestion, chunking, embedding, storage. Standard RAG with no customisation needed.
OpenSearch Serverless	Binary vectors and FP16, hybrid search, fine-grained access control. Sensitive documents needing tuning.
Aurora PostgreSQL + pgvector	SQL filtering and vector similarity in a single query. Choose when structured metadata and relationships matter as much as similarity, or Postgres is already in use.

SELECT * FROM cases WHERE jurisdiction = '9th Circuit' AND date > '2020-01-01' ORDER BY embedding <-> query_embedding LIMIT 10;

That single query does both the filter and the similarity ranking — in OpenSearch this would span two systems.

Multi-tenant architectures (SaaS)

Pattern	How	Trade-off
Silo	Separate index or database per tenant	Safest, most expensive
Pool	Shared store, filtered by <code>tenant_id</code>	Cheapest, depends entirely on the filter being correct
Bridge	Shared infrastructure, separate schemas	Middle ground

Law firms commonly mix: silo for large clients, pool for smaller ones. **In a pool model the filter must be applied inside the query**, not in application logic — one bug otherwise leaks client data.

Security controls

- **Encryption** — at rest via KMS, and in transit
- **Access control** — IAM plus fine-grained index/document/field-level control in OpenSearch
- **Redaction / masking** — **before embedding**. Once PII is embedded it lives inside the vector and is very hard to remove.

OpenSearch ML integration and cross-account

- OpenSearch can source embeddings from **Bedrock, SageMaker AI, Hugging Face models, or custom endpoints**; the Neural plugin wires this up so OpenSearch requests embeddings itself
- **Cross-account**: when Bedrock models sit in a different account from OpenSearch, use IAM role assumption plus resource policies on both sides

4. Advanced Search Architectures

Semantic vs hybrid

```
"Section 230"
  Semantic only -> also returns Section 220, Section 240
                  (the meanings are similar!)  -- WRONG

"consequences of breaking a contract"
  Keyword only  -> misses "breach of contract remedies"
                  (no shared words)                -- ALSO WRONG
```

Legal research needs both. An attorney searching a citation wants that exact citation, not something adjacent.

BM25 — the keyword half

- Established keyword ranking algorithm: term frequency in the document, term rarity across the corpus (rarer = more significant), and document length
- Beats vector search on **exact terms, citations, statute numbers and proper nouns**
- Hybrid blends the BM25 score with the vector similarity score

Configuring it in Bedrock

```
OverrideSearchType in KnowledgeBaseVectorSearchConfiguration
Values: SEMANTIC | HYBRID
```

Hybrid requires OpenSearch Serverless with filterable text fields. Not every vector store supports it — this constraint is exam-relevant.

Reranker models

```
Step 1: vector search -> top 100 chunks (fast, coarse)
Step 2: reranker      -> rescore those  -> top 10 (slow, precise)
```

Why a reranker is more accurate: in vector search, query and document are embedded *separately* and then compared — the model never saw them together. A reranker reads **query and document jointly**, so it judges relevance far better.

Why not use it alone: it's too slow to run across 15 million documents. Hence the two-stage design. Bedrock offers rerank models (Cohere Rerank, Amazon Rerank).

Legal-specific relevance signals

- **Citation network analysis** — a case cited frequently carries more authority. This comes from the **graph**, never from similarity.
- **Jurisdiction-aware filtering** — driven by query context and the attorney's profile
- **Precedential value** boost, as in Task 1.4

Full search pipeline

```
Query
-> hybrid search (vector + BM25)
-> metadata filter (jurisdiction, date, matter permissions)
-> top 100
-> reranker model
-> citation network boost
-> precedential value boost
-> top 10 -> FM
```

5. Query Handling Systems

Everything so far improved the *documents*. This section addresses the **query** itself — attorneys rarely phrase questions in a way that searches well.

Query expansion — using Bedrock

```
"non-compete enforceability"
  | Bedrock
"non-compete enforceability, restrictive covenants,
employee mobility restrictions, trade secret
protection, reasonable restraint of trade"
```

- The attorney typed two words; the relevant documents use different vocabulary entirely
- **Transcript: 23% improvement in recall**
- **Trade-off:** expand too aggressively and irrelevant results creep in

Query decomposition — using Lambda

"Can a non-compete be enforced in California if the employee relocated to Texas, and trade secrets are involved?" — that is **three questions**. Searched as one, the vector becomes an average of three concepts and answers none of them well.

1. California non-compete enforceability

2. Multi-state jurisdiction conflicts, employment

3. Trade secret exception to non-compete

Step Functions — parallel sub-query execution

```
Complex query
-> Lambda: decompose
-> Step Functions Parallel
    +-- sub-query 1 search
    +-- sub-query 2 search
    +-- sub-query 3 search
-> merge -> rerank -> FM
```

Transcript: 40% reduction in response time. Same Step Functions parallel pattern as Task 1.3, applied to sub-queries instead of data types.

Query embedding, Neptune Analytics, event-driven indexing

- **Bedrock Knowledge Bases embeds the natural-language query automatically.** For manual control, embed in Lambda and query the vector store directly.
- **Neptune Analytics** — graph database with vector search, algorithm `.vectors.topKByEmbedding`. Fits legal perfectly: "*find documents like this one **and** cited by the Supreme Court*" — similarity plus graph traversal in one place.
- **Event-driven indexing:** S3 events or DynamoDB Streams → **EventBridge Pipes** → Lambda → embed → vector store. Pipes carries filtering and enrichment inline, removing glue code.
- **Feedback loop:** attorney thumbs up/down reveals which queries fail, whether expansion helped, whether reranking ordered correctly. **Retrieval quality only improves when it is measured.**

6. Integration With Foundation Models

Mechanism	Who decides to retrieve	When to use
RAG	You — retrieve, then prompt	Simple, predictable, auditable
Function calling	The model — it requests what it needs	Multi-step questions, flexible
MCP	Standardised at the protocol level	Many tools or models to integrate consistently

RAG with citations

Requirement: **citations to source documents on every response**. Chunks are passed with their source metadata:

```
[Chunk 1] Source: Smith v. Jones, 9th Cir. 2021, p.14
[Chunk 2] Source: Cal. Bus. Code § 16600
```

The model cites them; the attorney can verify. **This requires maintaining a mapping between each vector and its original document** — without it, citation is impossible.

Function calling

```
"What does Smith v. Jones hold, and is it still good law?"

Model decides it needs two things:
-> search_cases("Smith v Jones")
-> check_overturned("Smith v Jones")
-> composes the answer from both
```

- You supply the tools; **the model chooses which and when**
- This is the mechanism underneath **Bedrock Agents** (action groups) — see Task 1.1
- **Trade-off:** flexible, but non-deterministic and harder to audit

Model Context Protocol (MCP)

- **Problem it solves:** every tool integration is custom code — each vector store has its own interface, each model its own function-calling format
- **MCP is a standard protocol** between models and data sources. An **MCP server** exposes data; an **MCP client** consumes it.
- **Analogy:** USB. Before it, every device had its own cable; afterwards, one standard fits everything.
- Build the MCP server once and any MCP-compatible model can use it

Standardised API patterns and logging

```
{ "results": [ {
  "content": "...",
  "source": { "doc_id": "...", "title": "...", "page": 14 },
  "metadata": { "jurisdiction": "9th Cir", "date": "2021" },
  "score": 0.89
} ] }
```

Same principle as the adapter pattern in Task 1.2 — one consistent shape regardless of what sits behind it.

Comprehensive logging is non-negotiable in legal: which query ran, which documents were retrieved, what prompt was assembled, what the model answered. If legal advice was built on an AI response, the audit trail must exist months later.

Data Preparation Pipeline

```
collect -> clean -> transform -> structure -> chunk
        -> embed -> store (with metadata + source mapping)
```

Ingestion flows typically start with document uploads to an S3 bucket configured as a Bedrock Knowledge Bases data source.

Numbers From This Scenario

- **92%** recall on relevant legal precedents
- **+23%** recall from query expansion
- **–40%** response time from parallel decomposed queries
- **–70%** research time; **+42%** precedents discovered; **95%** of attorneys report higher confidence